RFC - Metric Tracking System

Functional Requirements
- Problems

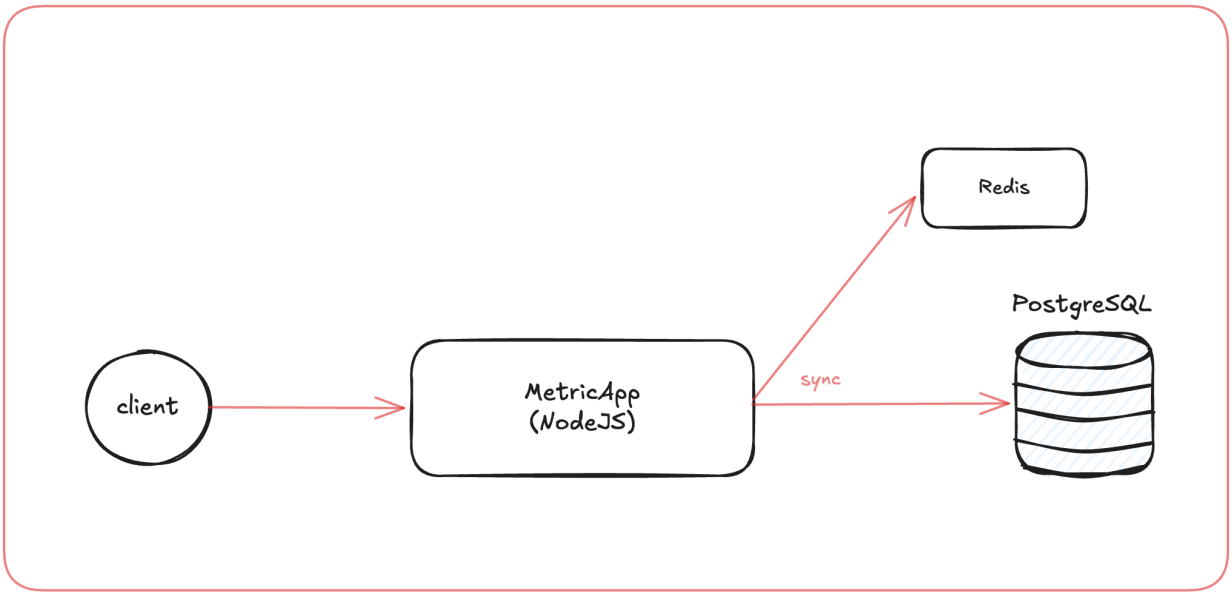
Non-Functional Requirements (Assumptions)
Architecture
- Deployment Model
- Layer Diagram

Implementation
- Database Schema
- APIs
- Cache Key Design
- SQS Message Format

Functional Requirements

- Users should be able to add a new metric with: Date, Value, Unit
- Users should be able to retrieve:
  - A list of all metrics based on type (Distance / Temperature)
  - Data for chart drawing:
    - Use the latest metric entry per day
    - Filter by metric type
    - Filter by a specific time period (e.g. 1 month, 2 months)
- If a specific unit is provided when calling the APIs, the system should convert and return the values in that unit.

Simple Architecture



Problems

Write side:

- Database becomes a bottleneck
- Database connection exhaustion
- Synchronous writes increase latency

Read side:

- Massive metric table growth
- Large index size
- Slow scans for time-range queries

Non-Functional Requirements (Assumptions)

Assumption

- 100M Daily Active Users (DAU)
- 1 activity per user per day
- Metric updated every 5 minutes → 12 updates/day

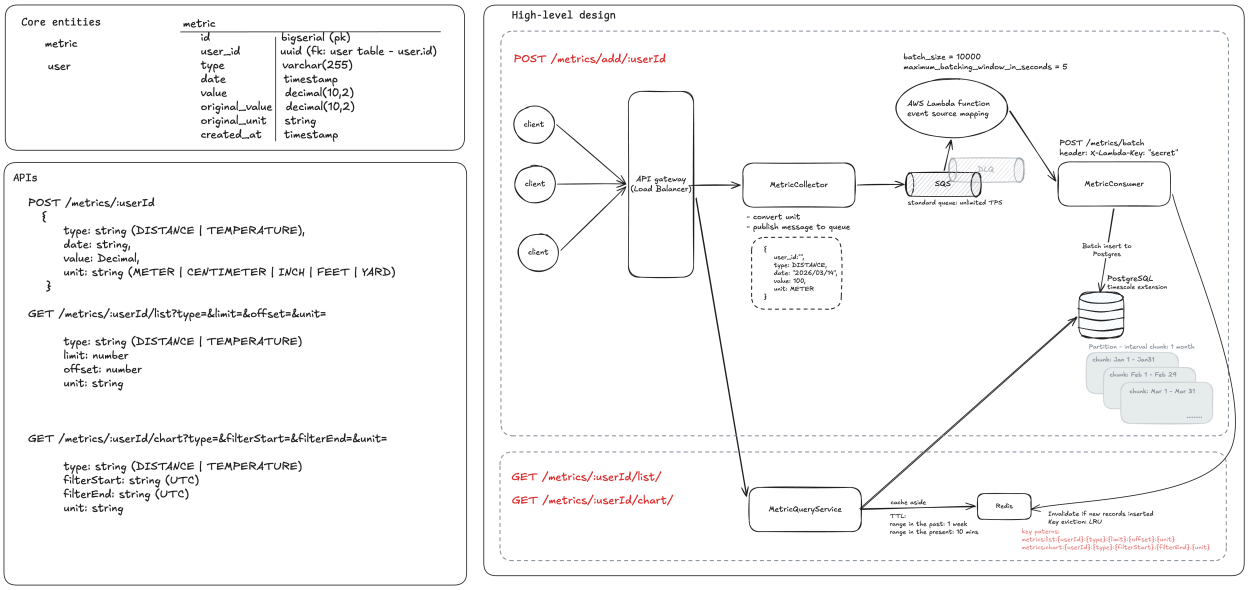
Write volume

- 100M activities × 12 updates
- ≈ 1.2B metric entries/day

Traffic

- ≈ 14k write RPS (average)

Architecture



The system separates **write** and **read** paths into **3 independently deployable containers** for independent scaling. Writes go through an async pipeline (SQS + batch consumer), reads go through a cache-aside layer (Redis).

Deployment Model

Multi-deployables with 3 separate Express servers, each with its own entry point.

Container	Service	Routes	Scaling Concern
`metric-collector`	MetricCollector	`POST /metrics/:userId`,	Scales with write throughput
`metric-query`	MetricQueryService	`GET /metrics/list/:userId`, `GET /metrics/chart/:userId`	Scales with read traffic

Container	Service	Routes	Scaling Concern
`metric-consumer`	MetricConsumer	`POST /metrics/batch` (internal used by lambda function)	Already receive batch requests from lambda - only idempotent concern

Layer Diagram

```
WRITE PATH (metric-collector container):
  Client
    ↓
  API Gateway (ALB)
    ↓
  MetricCollector (unit conversion → normalize to base unit,
publish to SQS)
    ↓
  SQS (standard queue)
    ↓
  MetricConsumer
    ↓
  POST /metrics/batch (internal endpoint, requests from lambd
a function)
    ↓
  PostgreSQL + TimescaleDB (hypertable with 1month chunks, au
to-retention)

READ PATH (metric-query container):
  Client
    ↓
  API Gateway
    ↓
  MetricQuery (business logic + unit conversion on output)
    ↓
  Redis
```

```
    ↓
  PostgreSQL
```

Implementation

Database Schema

`metrics`		
id	BIGSERIAL	NOT NULL
user_id	UUID	NOT NULL
type	VARCHAR(255)	NOT NULL
date	TIMESTAMPTZ	NOT NULL
value	DECIMAL(10,2)	NOT NULL
original_value	DECIMAL(10,2)	NOT NULL
original_unit	VARCHAR(20)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

DB: PostgreSQL + timescaledb extension

Chunk time interval: 1 month

Indexing: (user_id, type, date DESC)

Retention policy interval: 1 year

APIs

Current routes use query params for userId. Architecture uses path params.

Current	Method	New
`POST /api/metrics/:userId`	POST	Add metric
`GET /api/metrics/:userId/list?type=&unit=&limit=&offset=`	GET	Get paginated list of user's metric
`GET /api/metrics/:userId/chart?type=&unit=&filterStart=&filterEnd=`	GET	Get user's metric for chart data point

Current	Method	New
`POST /metrics/batch`	POST	Batch insert metrics to PostgreSQL - request from Lambda function (internal, X-Lambda-Key header)

Cache Key Design

```
metrics:list:{userId}:{type}:{limit}:{offset}:{unit} → list r
esults (unit is optional)
metrics:chart:{userId}:{type}:{filterStart}:{filterEnd}:{uni
t} → chart results (unit is optional)
```

- On batch insert: invalidate all keys matching `metrics:*:{userId}:{type}:*`

SQS Message Format

```
{
  "user_id": "e",
  "type": "distance",
  "date": "2026-03-14T10:30:00Z",
  "value": 100, // base unit
  "original_value": 328.08,
  "original_unit": "feet"
}
```